Middleware Implementation in ASP.NET Core

Name: Diksha Chaudhary

Technology: ASP.NET Core (.NET 10 Minimal Hosting Model)

IDE: Visual Studio Code (macOS)

Date: *(26/02/2026)*

---

1. Objective

The objective of this assignment is to understand and implement middleware in ASP.NET Core. Middleware components are used to handle requests and responses in the HTTP pipeline. In this assignment, middleware is used for logging, exception handling, security, HTTPS enforcement, and serving static files.

---

2. Introduction to Middleware

Middleware is a software component that is assembled into an application pipeline to handle requests and responses. Each middleware can perform operations before and after the next component in the pipeline.

In ASP.NET Core, middleware is configured inside the Program.cs file and executed in the order in which it is added.

---

3. Features Implemented

3.1 Request and Response Logging

A custom middleware is created to log:

- HTTP request method
- Request path
- Response status code.
- This helps in monitoring application behaviour and debugging.

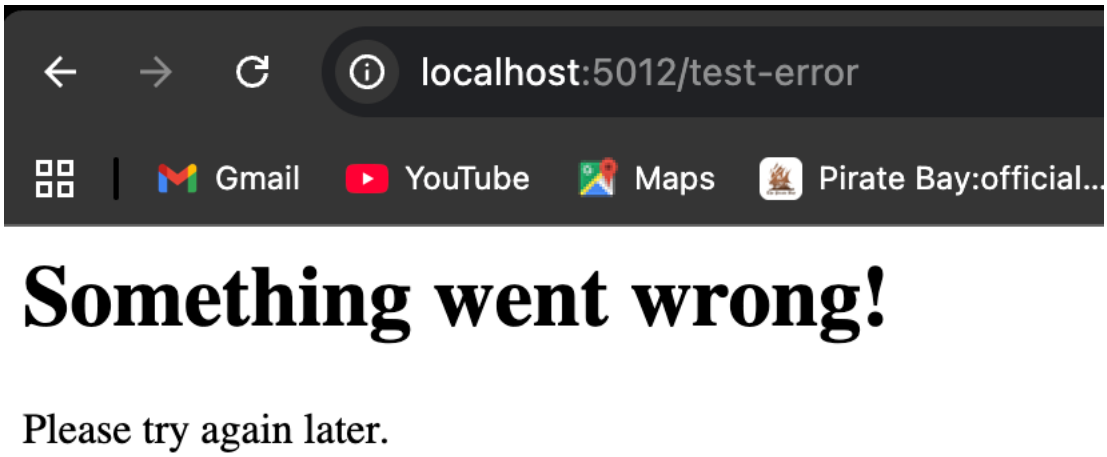
```
Request: GET /
Response Status Code: 200
Request: GET /favicon.ico
Response Status Code: 404
Request: GET /
Response Status Code: 200
Request: GET /index
Response Status Code: 404
Request: GET /test-error
Request: GET /error
```

```
(base) dikshachaudhary@Dikshas-MacBook-Air MiddlewareDemo % dotnet run
Using launch settings from /Users/dikshachaudhary/MiddlewareDemo/Properties/launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5012
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/dikshachaudhary/MiddlewareDemo
```

3.2 Global Exception Handling

A global exception handler is implemented using: app.UseExceptionHandler("/error");

Whenever an unhandled exception occurs, the application redirects to a custom error endpoint that displays a user-friendly error message instead of crashing.



3.3 HTTPS Redirection

The application forces secure communication using: app.UseHttpsRedirection();

This automatically redirects HTTP requests to HTTPS.

3.4 HSTS (HTTP Strict Transport Security)

HSTS is enabled for production environments to ensure the browser always communicates using HTTPS.

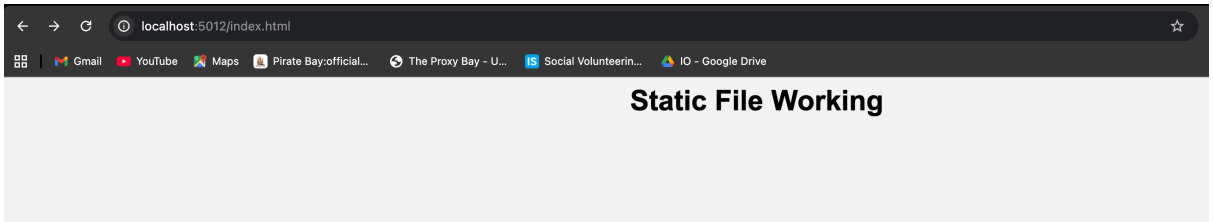
3.5 Serving Static Files

Static files such as:

- HTML
- CSS
- JavaScript

are stored in the wwwroot folder and served using: app.UseStaticFiles();

This allows the application to deliver frontend resources.



3.6 Content Security Policy (CSP)

A Content Security Policy header is added to protect the application from XSS attacks by allowing resources to load only from trusted sources.

---

4. Middleware Execution Flow

The middleware pipeline in this application follows this order:

1. Global exception handling
2. HSTS (production only)
3. HTTPS redirection
4. Static file middleware
5. Security headers (CSP)
6. Logging middleware
7. Endpoints

This order ensures proper error handling and secure request processing.

---

5. Testing and Output

The application was executed using: dotnet run

The following test cases were verified:

- Default route displaying the success message
- Static HTML page loading from the wwwroot folder
- Custom error page displayed when an exception occurs
- Request and response logs visible in the terminal

---

6. Result

All middleware components were successfully implemented and tested.

The application logs each request and response, handles exceptions globally, enforces HTTPS, serves static files, and applies security headers.

---

7. Conclusion

This assignment helped in understanding how middleware works in the ASP.NET Core request pipeline. It also demonstrated the importance of middleware ordering, security practices, and centralized exception handling in real-world applications.